

Multi-Task Learning for Computer Vision: Systematic Analysis of Optimizer-Batch Size Interactions

Rokibul Islam [Id: 2111088642]

Mobashshera Rahman [Id: 2212292642]

Al Nahian Niloy [Id: 2031445642]

1. PROJECT OVERVIEW

1.1 Title and Brief Description

Project Title: Multi-Task Deep Learning Framework: Joint Optimization of Image Classification, Object Detection, and Semantic Segmentation with Systematic Optimizer-Batch Size Analysis

Research Problem: Modern computer vision systems must perform multiple tasks simultaneously (e.g., autonomous vehicles classify traffic signs, detect pedestrians, and segment road surfaces). However, existing research analyzes optimization algorithms and batch sizes for single tasks in isolation, leaving critical questions unanswered: Which optimizers work best across multiple tasks? How do small batch sizes (8, 16, 32) — crucial for edge devices — affect multi-task learning? How should tasks share parameters for maximum efficiency?

Project Goals:

- Develop a multi-task learning (MTL) architecture with one shared encoder (VGG-style backbone) and three task-specific heads for classification, detection, and segmentation.
- Systematically evaluate 8 modern optimizers (SGD, RMSProp, Adam, AdamW, SparseAdam, Adamax, NAdam, RAdam) across 3 batch sizes (8, 16, 32).
- Conduct rigorous experiments with ≥ 3 independent runs per configuration, learning rate sweeps, and normalization alternatives (BatchNorm vs GroupNorm).
- Perform statistical analysis using 2-way and 3-way ANOVA, Tukey HSD, and effect size reporting (Cohen's d).
- Provide actionable recommendations for deploying multi-task systems in resource-constrained environments (edge computing, mobile devices).

Key Innovation: First comprehensive study analyzing optimizer-batch size-task interactions across three vision tasks simultaneously with rigorous statistical validation, addressing the growing need for efficient multi-task AI systems.

Real-World Impact:

- **Autonomous Vehicles:** One model classifies signs, detects obstacles, segments drivable areas (e.g., Tesla FSD, Waymo).
- **Medical Imaging:** Diagnose disease (classification), locate tumors (detection), delineate boundaries (segmentation).
- **Robotics:** Recognize objects (classification), find grasp points (detection), plan manipulation paths (segmentation).

Expected Outcomes:

- 144+ trained models (48 configurations $\times$ 3 seeds minimum).
- Statistical identification of optimal optimizer-batch-task configurations.
- 60% parameter reduction vs. three separate models (25M vs 45M parameters).
- Practical deployment guidelines with memory-latency-accuracy trade-off analysis.
- Publication-quality research report (20–25 pages) and open-source codebase.

2. RELATED WORK & RESEARCH GAP

2.1 Recent Optimizer Studies (2024–2025)

Single-Task Optimizer Comparisons:

- Murthy et al. (2024) - "Optimizing Convolutional Neural Networks: A Comparative Analysis" (IEEE). Compared 7 optimizers (SGD, RMSProp, Adam, Adadelta, Adagrad, AdamW, NAdam) on CNN classification. *Key Finding*: NAdam achieved best accuracy (92.3% on CIFAR-10) but Adam converged fastest. *Gap*: Only classification task; no batch size analysis; no multi-task scenarios.
- Umeda & Iiduka (2024) - "Increasing Both Batch Size and Learning Rate Accelerates SGD" (TMLR 2025, arXiv:2409.08770). *Key Finding*: Linear scaling rule (double batch size $\rightarrow$ double LR) maintains accuracy while reducing training time. *Gap*: Focused on SGD only; theoretical emphasis; not tested on detection/segmentation.
- Syamsul et al. (2024) - "Optimizers Comparative Analysis on YOLOv8 and YOLOv11" (IEEE, Dec 2024). Compared Adam, AdamW, NAdam, RAdam on object detection (YOLOv11). *Key Finding*: RAdam achieved best mAP (62.1% on COCO) with most stable training. *Gap*: Detection-only; standard batch sizes (32–64); no classification/segmentation.
- Zakaria et al. (2023) - "Improved Image Classification Using Enhanced VGG Architecture" (JOIV, Vol 7). Improved VGG with batch normalization + global average pooling $\rightarrow$ 79% parameter reduction. *Gap*: Architecture focus; single task; no optimizer comparison.

2.2 Multi-Task Learning Research (2024–2025)

- Graham et al. (2022, updated 2024) - "One Model is All You Need: Multi-Task Learning" (Medical Image Analysis). Unified model for tumor classification + detection + segmentation. *Key Finding*: Shared encoder improved all tasks vs. separate models (3–5% accuracy gain). *Gap*: Medical domain-specific; no optimizer analysis; fixed hyperparameters.
- Sample-Centric MTL (2024) - "Multi-Task Learning for Detection and Segmentation" (arXiv:2510.13226). Proposed sample-centric loss weighting to balance task difficulties. *Key Finding*: Dynamic weighting improved mIoU by 4.2% on Cityscapes. *Gap*: Complex loss weighting; no batch size or optimizer study.
- IEEE Edge AI Benchmarking (2024) - "Edge Computing for High-Performance ML Inference". Tested small batch sizes (8–16) on NPUs and GPUs for edge devices. *Key Finding*: Batch size 8–16 optimal for memory-constrained devices (phones, IoT). *Gap*: Hardware focus; no multi-task or optimizer analysis.

2.3 Identified Research Gaps

Research Area	Existing Work	Gap (This Project Addresses)
Optimizer Studies	Single-task classification	No detection/segmentation (Tests 8 optimizers across 3 tasks)
Batch Size Analysis	Standard sizes (32–256)	No small batches (8–32) for edge (Focuses on 8,16,32)
Multi-Task Learning	Architecture design	No optimizer-batch analysis (Systematic optimizer-batch-task study)
Statistical Rigor	Single runs, p-values only	No multi-seed, effect sizes (3 seeds, ANOVA, Cohen’s d)
Cross-Task Interactions	Tasks studied separately	No shared encoder analysis (Measures knowledge transfer quantitatively)

3. PROPOSED METHODOLOGY

3.1 Multi-Task Architecture Design

Shared Encoder (VGG-Style Backbone):

Input: $x \in (224 \times 224 \times 3)$

Encoder (x ; $_shared$):

Block 1: $\text{Conv}(3 \rightarrow 64, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(64 \rightarrow 64, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$
Output: $h1 \in (112 \times 112 \times 64)$

Block 2: $\text{Conv}(64 \rightarrow 128, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(128 \rightarrow 128, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$
Output: $h2 \in (56 \times 56 \times 128)$

Block 3: $\text{Conv}(128 \rightarrow 256, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(256 \rightarrow 256, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$
Output: $h3 \in (28 \times 28 \times 256)$

Block 4: $\text{Conv}(256 \rightarrow 512, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(512 \rightarrow 512, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$
Output: $h4 \in (14 \times 14 \times 512)$

Block 5: $\text{Conv}(512 \rightarrow 512, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(512 \rightarrow 512, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$
Output: $f = h5 \in (7 \times 7 \times 512)$ ← Shared features for all tasks

$_shared$: ~14M parameters (shared across tasks)

Task-Specific Heads:

Classification Head $_cls(f; _cls)$:

$\text{GlobalAveragePool}(f) \rightarrow z \in 512$
 $\text{FC}(512 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.4)$
 $\text{FC}(256 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.4)$
 $\text{FC}(128 \rightarrow 10) \rightarrow \text{Softmax} \rightarrow p_cls \in 10$

$_cls$: ~0.3M parameters

Output: Class probabilities for CIFAR-10 (10 classes)

Detection Head $_det(f; _det)$:

Region Proposal Network (RPN):

- Anchor generation: 9 anchors per location (3 scales $\times$ 3 ratios)

- Conv(512→512, 3×3) → ReLU
- Objectness: Conv(512→18, 1×1) → Sigmoid [2 × 9 anchors]
- Box regression: Conv(512→36, 1×1) [4 × 9 anchors]

ROI Pooling: Crop features for each proposal → 7×7×512

Classification + Localization:

- FC(7×7×512→1024) → ReLU
- Class scores: FC(1024→21) [20 classes + background]
- Box refinement: FC(1024→84) [21 classes × 4 coords]

_det: ~5M parameters

Output: Bounding boxes $B = \{(x_i, y_i, w_i, h_i, c_i, s_i)\}$ for PASCAL VOC

Segmentation Head _seg(f; _seg):

Decoder (Upsampling):

- Upsample1: 7×7×512 → 14×14×512 (TransposeConv, stride=2)
+ Skip connection from h4 (14×14×512)
- Upsample2: 14×14×512 → 28×28×256 (TransposeConv, stride=2)
+ Skip connection from h3 (28×28×256)
- Upsample3: 28×28×256 → 56×56×128 (TransposeConv, stride=2)
+ Skip connection from h2 (56×56×128)
- Upsample4: 56×56×128 → 112×112×64 (TransposeConv, stride=2)
+ Skip connection from h1 (112×112×64)
- Upsample5: 112×112×64 → 224×224×64 (TransposeConv, stride=2)
- Final: Conv(64→21, 1×1) → Softmax per pixel → $M \wedge (224 \times 224 \times 21)$

_seg: ~6M parameters

Output: Pixel-wise mask for PASCAL VOC Segmentation (21 classes)

Total Parameters: 25M (vs 45M for three separate models = 60% reduction)

3.2 Multi-Task Loss Functions

Overall Objective:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_2 \mathcal{L}_{\text{det}} + \lambda_3 \mathcal{L}_{\text{seg}}$$

where $\lambda_1 = 0.3$, $\lambda_2 = 0.3$, $\lambda_3 = 0.4$ (segmentation weighted higher due to complexity).

Classification Loss (Cross-Entropy):

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(p_{ic})$$

Detection Loss (Localization + Classification):

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{cls-det}}$$

Localization Loss (Smooth L1):

$$\mathcal{L}_{\text{loc}} = \frac{1}{N_{\text{pos}}} \sum_{i \in \text{pos}} \text{SmoothL1}(b_i - \hat{b}_i)$$

$$\text{SmoothL1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Classification Loss (Focal Loss for class imbalance):

$$\mathcal{L}_{\text{cls-det}} = -\frac{1}{N_{\text{pos}}} \sum_{i=1}^{N_{\text{anchor}}} \alpha_i (1 - p_i)^\gamma \log(p_i)$$

with α_i balancing weight, $\gamma = 2$.

Segmentation Loss (Pixel-Wise Cross-Entropy + Dice):

$$\mathcal{L}_{\text{seg}} = \mathcal{L}_{\text{CE-seg}} + \mathcal{L}_{\text{Dice}}$$

Pixel-wise CE:

$$\mathcal{L}_{\text{CE-seg}} = -\frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \sum_{c=1}^{21} y_{hwc} \log(p_{hwc})$$

Dice Loss:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_{c=1}^{21} \sum_{h,w} p_{hwc} y_{hwc}}{\sum_{c=1}^{21} \sum_{h,w} (p_{hwc} + y_{hwc})}$$

Benchmark Testing

This section lists a comprehensive set of benchmark tests, measurements and experimental variants to run (prioritize by available GPU hours). Each test entry includes **purpose**, **setup**, **metrics to record** and **notes**. Mark priorities as: (Core) = must-run, (Recommended) = high value, (Optional/Stretch) = if time allows.

Core (must-run) benchmarks

- **Single-task classification baseline (Core):** Train the CustomVGG on CIFAR-10 for each optimizer $\times$ batch configuration.
Purpose: Establish baseline accuracy, convergence and memory usage per optimizer and batch size.
Setup: 8 optimizers $\times$ 3 batch sizes $\times$ 3 seeds; epochs=50.
Metrics: train/val/test accuracy, loss curves, epochs-to-threshold, peak GPU memory, epoch time, throughput (samples/sec).
- **Normalization comparison (Core):** For small-batch settings (b=8,16) compare Batch-Norm vs GroupNorm.
Purpose: BN often degrades at small b; GroupNorm can be more stable.
Setup: Select top-4 optimizers from classification baseline, run BN vs GN for b=8,16.
Metrics: final accuracy, variance across seeds, loss curves.

Recommended benchmarks

- **Multi-task vs Single-task comparison (Recommended):** Train (i) shared MTL model (shared encoder + 3 heads) and (ii) three separate single-task models with comparable parameter budgets.
Purpose: Quantify parameter efficiency, performance trade-offs and transfer benefits.
Setup: Use top-4 optimizers, batch sizes 8/16/32; run 3 seeds per config.
Metrics: classification acc, mAP, mIoU, total params, inference latency, memory footprint.
- **Learning rate sweep (Recommended):** Small grid search of learning rates per optimizer to rule out LR-driven failures.

Purpose: Some optimizers are sensitive to LR; this avoids false conclusions.

Setup: For each optimizer test LR sets (Adam-family: [1e-4, 3e-4, 1e-3]; SGD: [1e-3, 3e-3, 1e-2]).

Metrics: best validation accuracy and corresponding LR, stability across seeds.

- **Gradient accumulation (Recommended):** Simulate larger effective batch sizes (e.g., effective $b=64$ via 4 accumulation steps of $b=16$).

Purpose: Compare real large-batch behavior vs accumulation, observe BN/optimizer differences.

Metrics: final accuracy, memory usage, training time overhead.

- **Optimizer warm-start / hybrid schedule (Recommended):** E.g., start with Adam/AdamW for initial epochs then switch to SGD for fine-tuning.

Purpose: Combine fast convergence with SGD generalization.

Setup: Test schedules like Adam (epochs 0–15) then SGD (epochs 16–50).

Metrics: final accuracy, convergence curve, stability.

Stretch benchmarks

- **Mixed precision training:** FP16 (AMP) for speed/memory.
Purpose: Determine if mixed precision changes relative optimizer rankings in resource-limited settings.
Metrics: throughput, memory, final accuracy drift.
- **Quantization and pruning :** Post-training quantization (INT8) and magnitude pruning followed by fine-tuning.
Purpose: Assess final model size vs accuracy trade-offs for edge deployment.
Metrics: model size (MB), inference latency (ms), accuracy drop.
- **Energy profiling:** Measure power consumption on a target device (e.g., Jetson Nano, Raspberry Pi + Coral).
Purpose: Practical energy/latency trade-offs for edge deployment.
Metrics: Watts, energy per inference (J), latency.
- **Robustness tests :** Evaluate against common corruptions (ImageNet-C style) and adversarial perturbations (FGSM/PGD).
Purpose: See whether optimizer/batch choices influence robustness.
Metrics: accuracy under noise, adversarial success rate.
- **Cross-dataset generalization :** Train on CIFAR-10 and evaluate transfer to CIFAR-100 subset or other small datasets.
Purpose: Determine if optimizer advantages are dataset-specific.

Profiling, monitoring and reproducibility

- **Profiling tools:** `torch.cuda.max_memory_allocated()`, `nvidia-smi` sampling, PyTorch profiler (cuda events), TensorBoard profiler.
- **Throughput measurement:** Measure wall-clock time for a full epoch and compute samples/sec using Python's `time` module.
- **Reproducibility checklist:** Log random seeds, PyTorch/CUDA/driver versions, deterministic flags, hardware specs. Save environment file (`requirements.txt` or `environment.yml`).
- **Automation:** Provide shell scripts / SLURM job templates or Google Colab notebooks for reproducible runs. Use W&B or TensorBoard for experiment tracking and artifact

storage.

Statistical benchmarking and reporting

- **ANOVA details:** For each dependent metric run 2-way ANOVA (Optimizer $\times$ Batch). If Phase includes Task as factor, run 3-way ANOVA (Optimizer $\times$ Batch $\times$ Task). Report F-statistics, p-values, and partial eta-squared for effect sizes.
- **Post-hoc tests:** Tukey HSD for pairwise comparisons; correct for multiple comparisons (Bonferroni or Holm) where appropriate.
- **Confidence intervals:** Report 95% CI for primary metrics using bootstrapping across seeds/runs.
- **Decision tables:** Construct tables mapping memory/latency constraints to recommended (optimizer, batch) pairs plus expected accuracy and uncertainty bands.

3.3 Optimizer Update Rules

(Brief equations for tested optimizers: SGD, Adam, AdamW, RAdam, NAdam, RMSProp, Adamax, SparseAdam.)

3.4 Datasets & Preprocessing

Task 1: Classification - CIFAR-10

- Size: 60,000 images (50K train, 10K test)
- Resolution: $32 \times 32 \times 3 \rightarrow$ resized to $224 \times 224 \times 3$
- Split: 45K train / 5K validation / 10K test
- Augmentation: RandomFlip, RandomCrop, ColorJitter, Normalize

Task 2: Detection - PASCAL VOC 2012

- Size: 17,126 images (11,540 train, 5,586 test)
- Use subset (5,000 images) for manageable training in Phase 2

Task 3: Segmentation - PASCAL VOC 2012 Segmentation

- Size: 2,913 images with pixel annotations (2,081 train / 832 val)

3.5 Experimental Design

Phase 1: Classification Only (select top optimizers for downstream phases).

Phase 2: Classification + Detection.

Phase 3: All Three Tasks.

Total runs estimated 162 (including LR sweeps and repeats).

3.6 Statistical Analysis Plan

- Descriptive statistics: mean $\pm$ std across seeds.
- Inferential: two-way ANOVA per task (Optimizer $\times$ Batch Size), three-way ANOVA (Optimizer $\times$ Batch Size $\times$ Task) for cross-task effects.
- Post-hoc: Tukey HSD, pairwise comparisons, Cohen’s d for effect sizes.
- Reproducibility measures: store seeds, framework versions, deterministic flags.

3.7 Evaluation Metrics

Classification: Top-1 Accuracy, Cross-Entropy Loss, Epochs to 85% accuracy.

Detection: mAP @ IoU=0.5, mAP @ IoU=0.75, per-class AP.

Segmentation: mIoU, Pixel Accuracy, Dice Score.

Efficiency: Training time/epoch, GPU memory, Inference latency, Model size.